

AGENTS AND PARALLEL WORK

Run parallel sessions with worktrees

Isolate parallel Claude Code sessions in separate git worktrees so changes don't collide. Covers the `--worktree` flag, subagent isolation, `.worktreeinclude`, cleanup, and non-git VCS hooks.

A **git worktree** is a separate working directory with its own files and branch, sharing the same repository history and remote as your main checkout. Running each Claude Code session in its own worktree means edits in one session never touch files in another, so you can have Claude building a feature in one terminal while fixing a bug in a second.

This page covers worktree isolation in the CLI. Everything below assumes a git repository. For other version control systems, see **Non-git version control**. The **desktop app** creates a worktree for every new session automatically.

Worktrees are one of several ways to run Claude in parallel. They isolate file edits, while **subagents** and **agent teams** coordinate the work itself. See **Run agents in parallel** to compare the approaches, or skip ahead to **Isolate subagents with worktrees** to use worktrees and subagents together.

Start Claude in a worktree

Pass `--worktree` or `-w` to create an isolated worktree and start Claude in it. By default, the worktree is created under `.claude/worktrees/<value>/` at your repository root, on a new branch named `worktree-<value>`:

```
claude --worktree feature-auth
```

To put worktrees somewhere else, configure a **WorktreeCreate hook**. Run the command again with a different name in another terminal to start a second isolated session:

```
claude --worktree bugfix-123
```

If you omit the name, Claude generates one such as `bright-running-fox`:

```
claude --worktree
```

You can also ask Claude to “work in a worktree” during a session, and it will create one with the `EnterWorktree` tool. Once in a worktree, Claude can switch directly to another one under `.claude/worktrees/` by calling `EnterWorktree` with the target path. The previous worktree stays on disk untouched.

Before using `--worktree` interactively in a directory for the first time, accept the workspace trust dialog by running `claude` once in that directory. If trust has not yet been accepted, `--worktree` exits with an error and prompts you to run `claude` in the directory first. Non-interactive runs with `-p` skip the **trust check**, so `claude -p --worktree` proceeds without it.

💡 Add `.claude/worktrees/` to your `.gitignore` so worktree contents don't appear as untracked files in your main checkout.

Choose the base branch

Worktrees branch from your repository's default branch, `origin/HEAD`, so they start from a clean tree matching the remote. If no remote is configured or the fetch fails, the worktree falls back to your current local `HEAD`. To always branch from local `HEAD` instead, set `worktree.baseRef` to `"head"` in **settings**. Setting `baseRef` to `"head"` makes new worktrees carry your unpushed commits and feature-branch state, which is useful when isolating subagents that need to operate on in-progress work. The setting accepts only `"fresh"` or `"head"`, not arbitrary git refs:

```
{
  "worktree": {
    "baseRef": "head"
  }
}
```

To branch from a specific pull request, pass the PR number prefixed with `#`, or a full GitHub pull request URL. Claude Code fetches `pull/<number>/head` from `origin` and creates the worktree at `.claude/worktrees/pr-<number>`:

```
claude --worktree "#1234"
```

For full control over how worktrees are created, configure a `WorktreeCreate hook`, which replaces the default `git worktree` logic entirely.

Copy gitignored files into worktrees

A worktree is a fresh checkout, so untracked files like `.env` or `.env.local` from your main repository are not present. To copy them automatically when Claude creates a worktree, add a `.worktreeinclude` file to your project root.

The file uses `.gitignore` syntax. Only files that match a pattern and are also gitignored are copied, so tracked files are never duplicated.

This `.worktreeinclude` copies two env files and a secrets config into each new worktree:

```
.worktreeinclude

.env
.env.local
config/secrets.json
```

This applies to worktrees created with `--worktree`, subagent worktrees, and parallel sessions in the desktop app.

Isolate subagents with worktrees

Subagents can run in their own worktrees so parallel edits don't conflict. Ask Claude to “use worktrees for your agents”, or set it permanently on a custom subagent by adding `isolation: worktree` to the frontmatter. Each subagent gets a temporary worktree that is removed automatically when the subagent finishes without changes.

Subagent worktrees use the same base branch as `--worktree`, so they branch from your repository's default branch unless `worktree.baseRef` is set to `"head"`.

Clean up worktrees

When you exit a worktree session, cleanup depends on whether you made changes:

- **No uncommitted changes, no untracked files, and no new commits:** the worktree and its

branch are removed automatically. If the session has a **name**, Claude prompts instead so you can keep the worktree for later

- **Uncommitted changes, untracked files, or new commits exist:** Claude prompts you to keep or remove the worktree. Keeping preserves the directory and branch so you can return later. Removing deletes the worktree directory and its branch, discarding any uncommitted changes, untracked files, and commits
- **Non-interactive runs:** worktrees created with `--worktree` alongside `-p` are not cleaned up automatically since there is no exit prompt. Remove them with `git worktree remove`

Worktrees that Claude created for subagents and **background sessions** are removed automatically once they are older than your `cleanupPeriodDays` setting, provided they have no uncommitted changes, no untracked files, and no unpushed commits. Worktrees you create with `--worktree` are never removed by this sweep.

While an agent is running, Claude runs `git worktree lock` on its worktree so that concurrent cleanup cannot remove it. The lock is released when the agent finishes. To clean up a worktree that the sweep keeps, run `git worktree remove`, adding `--force` if the worktree has uncommitted changes or untracked files.

Manage worktrees manually

For full control over worktree location and branch configuration, create worktrees with Git directly. This is useful when you need to check out a specific existing branch or place the worktree outside the repository.

Create a worktree on a new branch:

```
git worktree add ../project-feature-a -b feature-a
```

Create a worktree from an existing branch:

```
git worktree add ../project-bugfix bugfix-123
```

Start Claude in the worktree:

```
cd ../project-feature-a && claude
```

List your worktrees:

```
git worktree list
```

Remove one when you're done with it:

```
git worktree remove ../project-feature-a
```

See the [Git worktree documentation](#) for the full command reference. Remember to initialize your development environment in each new worktree: install dependencies, set up virtual environments, or run whatever your project's setup requires.

Non-git version control

Worktree isolation uses git by default. For SVN, Perforce, Mercurial, or other systems, configure

`WorktreeCreate` and `WorktreeRemove` **hooks** to provide custom creation and cleanup logic.

Because the hook replaces the default git behavior, `.worktreeinclude` is not processed when you use `--worktree`. Copy any local configuration files inside your hook script instead.

This `WorktreeCreate` hook reads the worktree name from stdin, checks out a fresh SVN working copy, and prints the directory path so Claude Code can use it as the session's working directory:

```

{
  "hooks": {
    "WorktreeCreate": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "bash -c 'NAME=$(jq -r .name);
DIR=\"$HOME/.claude/worktrees/$NAME\"; svn checkout
https://svn.example.com/repo/trunk \"$DIR\" >&2 && echo
\"$DIR\"'";
          }
        ]
      }
    ]
  }
}

```

Pair it with a `WorktreeRemove` hook to clean up when the session ends. See the [hooks reference](#) for the input schema and a removal example.

See also

Worktrees handle file isolation. The related pages below cover delegating work into those isolated checkouts and switching between the sessions you create:

- **Subagents**: delegate work to isolated agents within a session
- **Agent teams**: coordinate multiple Claude sessions automatically
- **Manage sessions**: name, resume, and switch between conversations
- **Desktop parallel sessions**: worktree-backed sessions in the desktop app